



# **ARES-01 Autonomous Teleoperated & Mission Rover Complete Technical & User Manual**

*A Comprehensive Project Manual, Build Guide &  
Technical Architecture*



**Barishal Govt. Polytechnic Institute  
Barishal, Bangladesh**

# Project Submission & Approval

*This project manual, entitled “**ARES-01: Autonomous & Teleoperated Mission Rover**,” has been prepared and submitted by the participating students of the Department of Electronics Technology, Barishal Govt. Polytechnic Institute, in partial fulfillment of the requirements for the Diploma in Engineering. This project reflects the combined efforts of the team in developing, integrating, testing, and documenting the ARES-01 robotic system.*

## **Submitted By:**

Department of Electronics Technology  
Diploma in Engineering  
Session: 2022-23

## **Supervised By:**

**Gazi Saiful Islam**

Chief Instructor, Electronics Department  
Barishal Govt. Polytechnic Institute  
Barishal, Bangladesh

# CERTIFICATE OF APPROVAL

This is to certify that the project entitled “**ARES-01: Autonomous & Teleoperated Mission Rover**” has been successfully carried out and documented by the students of the **Department of Electronics Technology, Barishal Govt. Polytechnic Institute**, under my supervision and guidance.

The project combines a four-wheel robotic rover, a five-degree-of-freedom robotic arm, an ESP32-S3-based control system, camera streaming, and a web-based control interface. The comprehensive project work encompasses hardware integration, electrical wiring, embedded firmware, software development, wireless communication setup, and full system-level integration.

To the best of my knowledge, the work presented in this documentation authentically reflects the project development carried out during the stated academic session. This manual has been prepared to thoroughly document the design, construction, operation, and technical implementation of the ARES-01 system.



Gazi Saiful Islam

Chief Instructor, Electronics Department

Barishal Govt. Polytechnic Institute, Barishal

# DECLARATION

We hereby declare that this project documentation entitled “**ARES-01: Autonomous & Teleoperated Mission Rover**” has been prepared based on the work carried out for the development, assembly, programming, and integration of the ARES-01 robotic system. The selection and integration of the electronic components, wiring, motor-control system, embedded firmware, web application, communication system, and robotic-arm integration were carried out as part of the project work.

A commercially available open-source Mini Mars Rover 3D-printable design was used as the mechanical reference/base for the rover chassis. The source of this external design has been acknowledged in the relevant section of this manual. All external references, open-source resources, and supporting materials used in the project have been identified wherever applicable.

We further declare that the information presented in this manual is intended to document the actual project implementation and known operating behavior of the ARES-01 system. Any information requiring further physical verification has not been intentionally represented as a confirmed specification.

# ACKNOWLEDGEMENT

First and foremost, we express our sincere gratitude to Almighty Allah for giving us the strength, patience, and opportunity to complete the ARES-01 Autonomous & Teleoperated Mission Rover project and prepare its complete technical documentation.

We would like to express our deepest appreciation to our project supervisor, Gazi Saiful Islam, Chief Instructor, Electronics Department, Barishal Polytechnic Institute, for his guidance, valuable suggestions, encouragement, and continuous support throughout the development of this project.

We are also grateful to the teachers and faculty members of the Department of Electronics Technology, Barishal Polytechnic Institute, for providing us with the academic foundation, laboratory knowledge, and technical guidance that helped us carry out this project.

Special thanks go to everyone who contributed directly or indirectly to the development of the rover, including those who assisted us during hardware assembly, wiring, testing, troubleshooting, and software development.

We would also like to acknowledge the open-source robotics community and the creators of the Mini Mars Rover 3D-printable design, which provided the mechanical reference used in the chassis development of ARES-01. The relevant source has been properly acknowledged in this manual.

Finally, we are thankful to our classmates, friends, and family members for their encouragement and support throughout the project work.

The completion of ARES-01 would not have been possible without the combined support, guidance, and encouragement received throughout the project.

# ABSTRACT

The **ARES-01 Autonomous & Teleoperated Mission Rover** is a multifunctional robotic platform developed to demonstrate the practical integration of mobile robotics, robotic manipulation, embedded control, wireless communication, and browser-based human-machine interaction.

## System Architecture & Hardware

The physical platform consists of a four-wheel drive rover chassis integrated with a 5-DOF robotic arm and an ESP32-S3-based control unit featuring video streaming capabilities:

**Locomotion & Manipulation:** Uses DC gear motors for rover locomotion and DC motors paired with H-bridge motor drivers for the 5-DOF robotic arm.

**Channel Expansion:** Two PCA9685 PWM controllers expand control channels and provide motor-control signals via an I2C interface.

**Wireless & Video:** Wi-Fi-enabled communication with HTTP and WebSocket protocols for low-latency command exchange and live camera streaming.

## Software & User Interface

The software architecture combines low-level embedded control with a modern web frontend:

**Firmware Layer:** An ESP32 firmware stack manages hardware interfacing, motor operation, camera streaming, and network requests.

**Web Application:** A React/Vite-based application provides an intuitive dashboard for manual movement control, arm joint positioning, and live feedback.

**Voice & AI Integration:** Supports voice commands in Bengali and English, with AI-assisted processing for advanced command interpretation.

## Scope & Document Purpose

ARES-01 serves as a functional robotics prototype and a platform for future enhancements, including closed-loop arm positioning, autonomous navigation, expanded sensor telemetry, and advanced safety mechanisms.

This manual documents the construction, hardware configuration, electrical connections, software environment, operating procedures, communication architecture, and troubleshooting routines of the system. It is designed to function both as an operational user guide and a technical reference for ongoing development and maintenance.

# Table of Contents

| Section                                            | Title / Description                            |
|----------------------------------------------------|------------------------------------------------|
| Part 1: User / Operation Manual                    |                                                |
| 1.1                                                | Downloading the Project                        |
| 1.2                                                | Setting Up the Environment (Node.js & pnpm)    |
| 1.3                                                | Running the Web Dashboard                      |
| 1.4                                                | Operating the Rover (Voice & Sliders)          |
| Part 2: Build / Assembly Manual & Hardware Details |                                                |
| 2.1                                                | 3D Printed Chassis (Rover Body)                |
| 2.2                                                | Detailed Component List & Usage Justification  |
| 2.3                                                | System Power Distribution (11.1V & 5V Logic)   |
| 2.4                                                | I2C Logic Level Translation                    |
| 2.5                                                | Comprehensive Motor Driver Wiring (Pin-to-Pin) |
| Part 3: Technical / Developer Manual               |                                                |
| 3.1                                                | Software & System Architecture                 |
| 3.2                                                | WebSocket Communication JSON Protocol          |
| 3.3                                                | Version Control & Troubleshooting Matrix       |
| 3.4                                                | Official GitHub Repository                     |

# Part 1: User / Operation Manual

This section is designed for absolute beginners. It explains exactly how to run the ARES-01 dashboard on your computer from scratch.

## 1.1 Downloading the Project

1. Open your web browser and go to the official GitHub repository:

None

[https://github.com/syedzayed-i2p/Ares\\_01](https://github.com/syedzayed-i2p/Ares_01)

2. Click on the green **<> Code** button located at the top right of the file list.
3. Select **Download ZIP**.
4. Once downloaded, locate the **.zip** file in your Downloads folder. Right-click on it and select **Extract All...** (or use WinRAR/7-Zip) to extract the files into a normal folder.

## 1.2 Setting Up the Environment (Node.js & pnpm)

Before running the dashboard, your computer needs Node.js.

1. Go to [nodejs.org](https://nodejs.org) and download the **LTS (Long Term Support)** version. Install it like any normal software.
2. Once Node.js is installed, we need to install **pnpm** (a fast package manager).
3. Press **Windows** Key, type **PowerShell**, right-click on it, and select **"Run as Administrator"**.

Type the following command and press Enter:

Shell

```
npm install -g pnpm
```

*Wait for the installation to finish.* You are now ready to run the project.

## 1.3 Running the Web Dashboard

1. Go to the extracted **Ares\_01** folder. Double click and go inside the **web\_app** folder.
2. Inside the **web\_app** folder, click on the **address bar** at the top of the folder window, type the command below, and hit **Enter** to open PowerShell directly in that folder:

Shell

```
powershell
```

First, download all the required project files by typing:

Shell

```
pnpm install
```

Once it finishes downloading, start the local server by typing:

Shell

```
pnpm run dev
```

You will see a message saying a local server has started. Open Google Chrome (or any modern browser) and navigate to the address below:

None

```
http://localhost:5173
```

## 1.4 Mission Control Dashboard Guide (How to Use)

Once the web app is open, follow these steps to connect and control the rover:

### 1. System Settings & Connecting to the Rover (⚙️ Icon)

- a. Look at the top-right corner of the dashboard and click the **Settings** (⚙️ Gear) icon.
- b. A modal titled **SYSTEM SETTINGS** will appear. Under "Hardware Connections", find the **Rover IP Address** field.
- c. Enter your Rover's local IP Address (e.g., 192.168.4.1 for Offline AP) and click **Connect**.
- d. The "ROVER OFFLINE" red badge at the top-left will turn into a green "ROVER ONLINE" badge, and telemetry (Ping and RSSI) will activate.

### 2. Manual Control Tab (Arm & Drive)

- a. **Drive Controls (Directional Pad)**: Located at the bottom right. Use the Up, Down, Left, and Right arrow buttons to move the rover. The red square in the middle acts as a quick STOP button.
- b. **5-DOF Arm Control**: Use the sliders or the increment/decrement arrows to rotate the Base, Shoulder, Elbow, Wrist, and Gripper. You can also manually type an exact coordinate angle (e.g., 90°) and click **EXECUTE**.

### 3. Offline Macros & AI Control Tab

- a. **Offline Text Command (AI Directive)**: Click the "OFFLINE MACROS" or "AI Directive" tab. Type commands manually like *"Initiate pick up sequence"* in the text box and hit send. The local AI engine will process the text offline without needing the cloud.
- b. **Voice Control**: Switch to the **Voice Command** tab. Click the language dropdown and select your language (e.g., বাংলা). Click the large **Mic** (🎤) button and speak (e.g., "সামনে যাও"). The transcript and system response will appear in the Voice Commands Log.

### 4. ESP Studio & Serial Monitor (Terminal >\_ Icon)

- a. Click the **Terminal (>\_)** icon at the top right to open the built-in **ESP Studio**.
- b. Here you can browse the embedded C++ firmware (like `main.cpp`) directly from the browser.

- c. To debug the rover, connect the ESP32 via USB and click the **Connect Serial** button on the right panel (Baud Rate: 115200) to view real-time hardware logs.

5. **UI Theme Customization (☾ / ☀️ Icon)**

Click the **Moon/Sun** icon at the top-right corner to toggle the dashboard's visual theme between **Dark Mode**, **Light Mode**, or System Default for comfortable viewing in any environment.

## Part 2: Build / Assembly Manual & Hardware Details

### 2.1 3D Printed Chassis (Rover Body)

The physical structure of ARES-01 is based on a highly robust open-source 3D printable model. If anyone wishes to replicate this rover entirely from scratch, they can easily download the chassis components and 3D print them (PLA or PETG recommended).

1. **Download Link:** <https://www.printables.com/model/791157-mini-mars-rover>

**Integration:** This chassis perfectly houses the 4 DC Rover Wheels and provides a stable mounting platform for the 5-DOF Robotic Arm Edge Kit on top.

### 2.2 Detailed Component List & Usage Justification

Every component in ARES-01 serves a specific purpose in the architecture.

| Sl | Component Name                                 | Qty | Specific Usage in Project                                                                                                                                                |
|----|------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | ESP32-S3 CAM Dev Board (with OV2640 & Antenna) | 1   | The absolute "Brain" of the rover. It processes the camera video, hosts the Wi-Fi WebSocket server, and issues I2C commands to move motors.                              |
| 2  | PCA9685 PWM Driver                             | 2   | Since the ESP32 doesn't have enough PWM pins for 9 motors, these drivers expand the capability. They convert simple I2C signals into 32 channels of perfect PWM signals. |

| SI | Component Name                     | Qty | Specific Usage in Project                                                                                                                        |
|----|------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 3  | I2C Logic Level Converter          | 1   | The ESP32 runs at 3.3V, but the PCA9685 prefers 5V logic. This chip bridges them, ensuring signals aren't corrupted or hardware fried.           |
| 4  | TB6612FNG Motor Driver             | 2   | Used for the smaller precision gear motors of the robotic arm (Shoulder, Elbow, Wrist, Gripper). Highly efficient for precise 4-joint control.   |
| 5  | L298N Motor Driver                 | 3   | Used for the heavy-duty rover base wheels (2 drivers) and the main base rotation joint of the robotic arm (1 driver) which require high current. |
| 6  | Voltage Regulator (Buck Converter) | 1   | Takes the dangerous 11.1V from the LiPo battery and converts it into a perfectly safe, stable 5.0V to power the delicate ESP32 and logic boards. |
| 7  | LiPo Battery (3300mAh, 11.1V, 3S)  | 1   | The main fuel tank. 11.1 Volts provides immense torque to the motors, while 3300mAh ensures long mission duration.                               |
| 8  | T-Connector (XT60)                 | 1   | A secure, spark-proof connector ensuring the high-current battery doesn't accidentally disconnect during rover movement.                         |
| 9  | Switch                             | 1   | The main kill-switch to power the entire system on or off instantly.                                                                             |
| 10 | Robotic Arm Edge Kit (5 DOF)       | 1   | The mechanical chassis of the arm, allowing complex object manipulation.                                                                         |
| 11 | DC Motor (Rover Wheel)             | 4   | The 4-wheel drive system enabling all-terrain navigation.                                                                                        |

## 2.3 System Power Distribution (11.1V & 5V Logic)

**WARNING:** Power distribution requires careful isolation between high-voltage motor supply and sensitive 5V/3.3V microcontrollers to prevent logic board failure.

| Power Rail      | Source          | Connected Components / Function                                                             |
|-----------------|-----------------|---------------------------------------------------------------------------------------------|
| 11.1V Raw Power | 3S LiPo Battery | Powers L298N and TB6612FNG motor driver power pins for high-torque wheel and arm actuation. |
| 5.0V Regulated  | Buck Converter  | Powers ESP32-S3, PCA9685 PWM drivers, and high-voltage side of I2C logic level translator.  |

## 2.4 I2C Logic Level Translation

Because the ESP32-S3 operates at 3.3V and the PCA9685 operates best at 5V, data must pass through the Logic Converter safely.

| ESP32-S3 (3.3V Logic) | Level Converter Bridge | PCA9685 (5V Logic)  |
|-----------------------|------------------------|---------------------|
| GPIO 1 (SDA)          | → LV1 → HV1            | → SDA (Board 1 & 2) |
| GPIO 2 (SCL)          | → LV2 → HV2            | → SCL (Board 1 & 2) |
| 3.3V Pin              | → LV (Reference)       | -                   |
| GND                   | → GND                  | → GND (Common)      |

## 2.5 Comprehensive Motor Driver Wiring (Pin-to-Pin)

To control all 9 DC motors (4 wheel motors and 5 arm motors), two PCA9685 16-channel PWM drivers are connected in parallel on the high-voltage side of the I2C bus. The complete mapping below matches the exact firmware pin definitions implemented in `HardwareController.cpp`.

### A. PCA9685 Addressing & Daisy-Chain Setup

#### 1. Hardware Addressing:

Each PCA9685 board on the same I2C bus must have a unique hardware address configured via the address solder jumpers (A0–A5):

**IMPORTANT NOTE:**

**Board 1 (Rover Wheels & Arm Base):** Keep as default address **0x40** (all address pads open).

**Board 2 (Robotic Arm):** Change to **0x41**. (To do this: Solder the two halves of the A0 pad together on the board).

| Module     | I2C Address | Jumper Config             | Assigned Function                            |
|------------|-------------|---------------------------|----------------------------------------------|
| PCA9685 #1 | 0x40        | Default (All Open)        | Rover Wheels & Arm Base                      |
| PCA9685 #2 | 0x41        | Bridge A0 (Solder Pad A0) | Arm Joints (Shoulder, Elbow, Wrist, Gripper) |

**2. Daisy-Chain Physical Wiring:**

To daisy-chain Board 1 and Board 2, connect the output header pins of Board 1 directly to the input header pins of Board 2 according to the table below:

| Signal / Pin | Board 1 Connection | Board 2 Connection | Description                             |
|--------------|--------------------|--------------------|-----------------------------------------|
| SCL          | SCL Output         | SCL Input          | Shares I2C clock line                   |
| SDA          | SDA Output         | SDA Input          | Shares I2C data line                    |
| VCC          | VCC                | VCC                | 5V Logic Power from Buck Converter      |
| GND          | GND                | GND                | Common System Ground                    |
| V+           | V+                 | V+                 | Optional external power rail connection |

## B. PCA Board 1 (0x40) → 2x L298N (Drive Wheels)

*Used exclusively for the 4-wheel drive system.*

| Motor Driver     | Target Motor      | PWM (Speed)      | Direction 1      | Direction 2     |
|------------------|-------------------|------------------|------------------|-----------------|
| L298N #1 (Left)  | Front Left Wheel  | PCA1 Ch 0 → ENA  | PCA1 Ch 2 → IN1  | PCA1 Ch 1 → IN2 |
|                  | Back Left Wheel   | PCA1 Ch 5 → ENB  | PCA1 Ch 4 → IN3  | PCA1 Ch 3 → IN4 |
| L298N #2 (Right) | Front Right Wheel | PCA1 Ch 6 → ENA  | PCA1 Ch 8 → IN1  | PCA1 Ch 7 → IN2 |
|                  | Back Right Wheel  | PCA1 Ch 11 → ENB | PCA1 Ch 10 → IN3 | PCA1 Ch 9 → IN4 |

*(Note: PCA1 Channel 15 is pulled HIGH in firmware to optionally enable TB6612 STBY pins if shared).*

## C. PCA Board 2 (0x41) → 1x L298N & 2x TB6612FNG (Robotic Arm)

*Used exclusively for the 5-DOF Robotic Arm.*

| Motor Driver     | Target Arm Joint | PWM (Speed)      | Direction 1       | Direction 2       |
|------------------|------------------|------------------|-------------------|-------------------|
| L298N #3 (Heavy) | Base Rotation    | PCA2 Ch 12 → ENA | PCA2 Ch 13 → IN1  | PCA2 Ch 14 → IN2  |
| TB6612 #1        | Arm Wrist        | PCA2 Ch 0 → PWMA | PCA2 Ch 1 → AIN1  | PCA2 Ch 2 → AIN2  |
|                  | Arm Elbow        | PCA2 Ch 3 → PWMB | PCA2 Ch 4 → BIN1  | PCA2 Ch 5 → BIN2  |
| TB6612 #2        | Arm Shoulder     | PCA2 Ch 6 → PWMA | PCA2 Ch 7 → AIN1  | PCA2 Ch 8 → AIN2  |
|                  | Arm Gripper      | PCA2 Ch 9 → PWMB | PCA2 Ch 15 → BIN1 | PCA2 Ch 11 → BIN2 |

# Part 3: Technical / Developer Manual

This section provides low-level developer documentation for the ARES-01 software ecosystem, system tasks, firmware architecture, and communication protocols.

## 3.1 Software & System Architecture

The system relies on a distributed architecture separating user interaction, control logic, and multi-core embedded processing.

### A. Frontend Application

Built with React, Vite, and Tailwind CSS. The web dashboard communicates with the ESP32-S3 over standard WebSocket connections for bi-directional command processing and receives an HTTP MJPEG stream for real-time video transmission.

### B. Firmware & FreeRTOS Core Allocation

The ESP32-S3 firmware utilizes FreeRTOS to allocate processing tasks across both Tensilica LX7 microcontroller cores to prevent camera streaming bottlenecks from interrupting motor signal timing:

5. **Core 0 (Protocol & Video Stream):** Dedicated to hosting the HTTP server, capturing OV2640 camera frames, MJPEG streaming, and handling Wi-Fi network traffic.
6. **Core 1 (Control & Hardware I2C):** Executes the WebSocket command parser, motor control logic, safety timeout monitors, and I2C signal transmission to both PCA9685 PWM drivers.

## 3.2 WebSocket Communication JSON Protocol

Commands sent from the web application to the ESP32-S3 are formatted as structured JSON packets over WebSocket connections on port 81.

### A. Rover Movement Payload

Specifies the directional action and drive speed percentage (0 to 100):

```
JSON
{
  "type": "rover",
```

```

    "action": "forward",
    "speed": 80
  }

```

Supported actions: "forward", "backward", "left", "right", and "stop".

## B. Arm Movement Payload

Specifies individual joint positions (angles from 0° to 180° or control intensity values):

```

JSON
{
  "type": "arm",
  "joint": "shoulder",
  "angle": 45,
  "speed": 100
}

```

Supported joint identifiers: "base", "shoulder", "elbow", "wrist", and "gripper".

## 3.3 Version Control & Troubleshooting Matrix

| Issue / Fault Symptom             | Probable Cause                | Diagnostic Check               | Corrective Action                                  |
|-----------------------------------|-------------------------------|--------------------------------|----------------------------------------------------|
| <b>Motors not responding</b>      | PWM Jumper still present      | Inspect L298N ENA/ENB headers  | Remove black jumper caps from L298N                |
| <b>ESP32 reboots continuously</b> | Power conflict on 5V pin      | Check L298N 5V-EN jumper       | Remove 5V-EN jumper cap on all drivers             |
| <b>Camera stream failure</b>      | GPIO pin conflict             | Check connection to GPIO 4 / 5 | Disconnect any peripherals on pins 4/5             |
| <b>Erratic motor movement</b>     | Common ground loop/potentials | Check GND continuity bus       | Tie all module (Driver, ESP32, Buck) GNDs together |
| <b>pnpm command not found</b>     | Node.js path error            | Open PowerShell, type node -v  | Re-install Node.js and check "Add to PATH"         |

### 3.4 Official GitHub Repository

All source code is maintained openly.

- **Repository:** [https://github.com/syedzayed-i2p/Ares\\_01](https://github.com/syedzayed-i2p/Ares_01)
- **License:** MIT Open Source

*End of Documentation. Developed by the ARES-01 Project Team.*